Exercise Sheet 10

The War Table

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (venv active).
- One file per exercise in `my_work/`, run from the lesson folder: `python my_work/ex1_head_count.py`.
- Every exercise starts with the **opener** (type it, don't paste it):

```
import pandas as pd

pd.set_option("display.width", 120)
quests = pd.read_csv("data/quests.csv")
```

- Printed tables must match exactly, including the row order and the little `Name:` / `dtype:` lines pandas adds under a Series.
- Hand in: commit and push to `lesson-10`, then open a pull request. Tell Lars which PivotTable you'll never build by hand again.

Exercise 1 Counting heads

5 min · *ex1_head_count.py*

> **Raven from Captain Siv (Quest Master):** *"Before I plan anything: how many quests per region, how often do parties come home victorious, and which monsters do we see most? It's three COUNTIFs. I have done them on my fingers. I ran out of fingers."*

(a) Print `quests["region"].value_counts()`. Empty line.

(b) The outcomes as shares: `value_counts(normalize=True)`, rounded to 3 dp. Empty line.

(c) How many different monsters there are (`.nunique()`), then the three most common.

EXPECTED OUTPUT

```
region
Thornwood         90
Ironhills         85
Saltmarsh         82
Northern Wilds    75
Greyfen           68
Name: count, dtype: int64

outcome
Success    0.648
Fled       0.205
Failed     0.148
Name: proportion, dtype: float64

Monsters: 8
monster
Goblins    81
Bandits    72
Wolves     64
Name: count, dtype: int64
```

Hint: `value_counts()` is a PivotTable with one row field and "Count" in the values. Sorted biggest first, always.

Exercise 2 The PivotTable

10 min · `ex2_the_pivottable.py`

> **Decree from Queen Aldara:** *"The average reward on a notice, by region. And the count, median and largest while you're there. And which region has cost the guild's strongroom the most. One table, please, not four."*

After the opener: `by_region = quests.groupby("region")["reward"]`.

(a) Print `by_region.mean().round(0)`. Empty line.

(b) Print `by_region.agg(["count", "mean", "median", "max"]).round(0)`. Empty line.

(c) The crowns actually paid out per region: group by region, sum `paid`, sort largest first, print the top three. Empty line.

(d) Print which region paid out most and which has the richest notices on average (`.idxmax()`).

EXPECTED OUTPUT

```
region
Greyfen           502.0
Ironhills         662.0
Northern Wilds    732.0
Saltmarsh         480.0
Thornwood         561.0
Name: reward, dtype: float64

                count   mean  median   max
region
Greyfen            68  502.0   450.0  2510
Ironhills          85  662.0   550.0  3530
Northern Wilds     75  732.0   610.0  4540
Saltmarsh          82  480.0   390.0  2960
Thornwood          90  561.0   405.0  2530

region
Ironhills         34730
Northern Wilds    33600
Thornwood         32780
Name: paid, dtype: int64

Most crowns paid out: Ironhills
Richest notices: Northern Wilds
```

Hint: `groupby("x")["y"]` says "row field x, values y". What comes after it (`.mean()`, `.sum()`, `.agg([...])`) is the "Summarize values by" box.

Exercise 3 Two row fields

10 min · `ex3_two_row_fields.py`

> **Raven from Captain Siv:** *"Which monsters beat us? Wins, flights and failures per monster, side by side, and the success rate, worst first. I want to know which notices to stop pinning up."*

(a) Add `quests["won"] = quests["outcome"] == "Success"`. The mean of a True/False column is the share of Trues: print the success rate by difficulty, times 100, rounded to 0 dp. Empty line.

(b) `both = quests.groupby(["monster", "outcome"])["quest_id"].count()`. Print the first six rows. Empty line.

(c) `table = both.unstack(fill_value=0)` turns the outcome level into columns. Print it. Empty line.

(d) Add `success_pct` = Success ÷ all quests of that monster × 100, rounded to 1 dp (`table.sum(axis=1)` is the row total). Print the table sorted by `success_pct`, worst first.

EXPECTED OUTPUT

```
difficulty
1    88.0
2    75.0
3    62.0
4    57.0
5    42.0
Name: won, dtype: float64

monster  outcome
Bandits  Failed     10
         Fled       12
         Success    50
Ghouls   Failed      9
         Fled       19
         Success    33
Name: quest_id, dtype: int64

outcome        Failed  Fled  Success
monster
Bandits            10    12       50
Ghouls              9    19       33
Giant Rats          1     7       34
Giant Spiders       4     9       29
Goblins            12    14       55
Trolls              9     9       13
Wolves             10     9       45
Wyrm                4     3        0

outcome        Failed  Fled  Success  success_pct
monster
Wyrm                4     3        0          0.0
Trolls              9     9       13         41.9
Ghouls              9    19       33         54.1
Goblins            12    14       55         67.9
Giant Spiders       4     9       29         69.0
Bandits            10    12       50         69.4
Wolves             10     9       45         70.3
Giant Rats          1     7       34         81.0
```

Hint: Two keys need a **list**: `groupby(["a", "b"])`. And yes, that's the Wyrm at the top: seven tries, no wins. Biscuit has been asked not to mention it at dinner.

Exercise 4 pivot_table, the real one

10 min · `ex4_pivot_table.py`

> **Decree from Queen Aldara:** *"Can it look like an actual PivotTable? Regions down, outcomes across, grand totals? I have been on the throne a long time."*

(a) `quests.pivot_table(index="region", columns="outcome", values="quest_id", aggfunc="count", margins=True)`. Empty line.

(b) The average reward by monster (down) and difficulty (across), rounded to 0 dp. Empty line.

(c) Crowns per day in the field: add `quests["per_day"] = quests["paid"] / quests["days"]`, then a pivot_table with `index="monster"`, `values="per_day"`, `aggfunc="mean"`. Round to 1 dp and sort by `per_day`, best first.

EXPECTED OUTPUT

```
outcome         Failed  Fled  Success  All
region
Greyfen             15    15       38   68
Ironhills           18    15       52   85
Northern Wilds       9    18       48   75
Saltmarsh            5    20       57   82
Thornwood           12    14       64   90
All                 59    82      259  400

difficulty         1      2      3       4       5
monster
Bandits          NaN  567.0  557.0   534.0     NaN
Ghouls           NaN    NaN  696.0   731.0   935.0
Giant Rats     143.0  142.0  180.0     NaN     NaN
Giant Spiders    NaN    NaN  887.0   793.0   836.0
Goblins        304.0  306.0  316.0   430.0     NaN
Trolls           NaN    NaN    NaN  1336.0  1280.0
Wolves           NaN  350.0  347.0     NaN     NaN
Wyrm             NaN    NaN    NaN     NaN  3130.0

               per_day
monster
Trolls            79.0
Giant Spiders     75.0
Bandits           55.6
Ghouls            54.2
Wolves            37.6
Goblins           34.8
Wyrm              34.6
Giant Rats        19.7
```

Hint: `margins=True` is the "Grand Total". The empty cells (NaN) are fights that never happened: nobody sends a party after a Wyrm at difficulty 1. And look where the Wyrm lands per day: the biggest notices, and almost nothing to show for them.

Exercise 5 XLOOKUP

10 min · `ex5_xlookup.py`

> **Raven from Orrin of the Clockworks:** *"Each region has a warden who wants a raven when a party passes through. Their list is in `data/regions.csv`. Quartermistress Pell XLOOKUPs it into the quest list by hand. That's 400 XLOOKUPs. One region is missing from the list. It always is."*

Opener, then `regions = pd.read_csv("data/regions.csv")`.

(a) Print `regions`. Empty line.

(b) `merged = quests.merge(regions, on="region", how="left")`. Print its shape and the first three rows of quest_id, region, danger, warden. Empty line.

(c) How many quests have no warden (`.isna().sum()`)? Which region are they in (`sorted(... .unique())`)? Fill the gaps with `"(no warden)"` using `.fillna()` and print the count again. Empty line.

(d) Danger check: `per_region = quests.groupby("region")["reward"].agg(["count", "mean"]).round(0).reset_index()`, merged with just `regions[["region", "danger"]]` using `how="inner"`. Print it sorted by danger, most dangerous first, then how many regions there were before and after the merge.

EXPECTED OUTPUT

```
region  danger                   warden  travel_days
0         Greyfen       2        Warden Alder Pike            2
1       Ironhills       4  Warden Hild Stonebridge            4
2  Northern Wilds       5  Warden Corwin Northwind            6
3       Saltmarsh       3   Warden Nerys Saltmarsh            3

(400, 15)
  quest_id     region  danger                  warden
0    Q0001  Thornwood     NaN                     NaN
1    Q0002  Saltmarsh     3.0  Warden Nerys Saltmarsh
2    Q0003  Thornwood     NaN                     NaN

90 quests without a warden
['Thornwood']
0 after fillna

           region  count   mean  danger
2  Northern Wilds     75  732.0       5
1       Ironhills     85  662.0       4
3       Saltmarsh     82  480.0       3
0         Greyfen     68  502.0       2
5 regions before the merge, 4 after
```

Hint: `how="left"` keeps every quest, NaN where there's no match (XLOOKUP's "#N/A"). `how="inner"` keeps only matches, so Thornwood silently disappears, all 90 quests of it. Count your rows after every merge.

Exercise 6 Reward bands

10 min · `ex6_reward_bands.py`

> **Raven from Quartermistress Pell:** *"Is a fat purse worth it? Cut the notices into four bands by reward and tell me how each band ends. Last time Dalvric did this the bands were 'small', 'medium', 'large' and 'Wyrm'."*

Opener, then `labels = ["Pittance", "Fair", "Handsome", "King's ransom"]`.

(a) `quests["band"] = pd.qcut(quests["reward"], 4, labels=labels)`. Print `value_counts().sort_index()`. Empty line.

(b) `pd.crosstab(quests["band"], quests["outcome"])`: counts. Empty line.

(c) The same with `normalize="index"`, times 100, rounded to 1 dp: how each band's quests end, in percent. Empty line.

(d) The reward range of each band: `groupby("band", observed=True)["reward"].agg(["min", "max"])`.

EXPECTED OUTPUT

```
band
Pittance         108
Fair              95
Handsome         100
King's ransom     97
Name: count, dtype: int64

outcome        Failed  Fled  Success
band
Pittance            9    20       79
Fair               18    13       64
Handsome           11    25       64
King's ransom      21    24       52
```

```
outcome        Failed  Fled  Success
band
Pittance          8.3  18.5     73.1
Fair             18.9  13.7     67.4
Handsome         11.0  25.0     64.0
King's ransom    21.6  24.7     53.6

               min   max
band
Pittance        90   310
Fair           320   470
Handsome       480   740
King's ransom  750  4540
```

Hint: `qcut` cuts into equal-sized groups; `cut` would cut into equal-width ranges. The bands aren't exactly 100 each because many notices share a round reward like 300 cr, and a tie can't be split.

Exercise 7 Who goes where STRETCH

15 min · `ex7_who_goes_where.py`

> **Raven from Captain Siv:** *"Somebody in the tavern said I only send the Bards on easy quests. I want to know if that's true before I say anything rude. And how is Wren Hawkridge doing? Everyone asks me. I never know."*

Opener, then read `data/heroes.csv` and `data/party.csv` as `heroes` and `party`. Each row of `party` is one hero on one quest.

(a) `trips = party.merge(heroes, ...).merge(quests, ...)`: two left merges, on `hero_id` then `quest_id`. Print `len(party)` and `len(trips)` (they must be equal). Empty line.

(b) Per class: number of trips, average difficulty, total loot. Use `.agg(trips=("quest_id", "count"), difficulty=("difficulty", "mean"), loot=("loot_share", "sum"))`. Round difficulty to 2 dp, sort by it, easiest first. Empty line.

(c) Pick two single cells out of that table with `.loc["Bard", "difficulty"]` (and the same for Warriors). Empty line.

(d) Wren Hawkridge: her number of quests and total loot (`,.2f`), then a pivot_table of her quests by outcome.

EXPECTED OUTPUT

```
1194 1194

         trips  difficulty      loot
class
Bard        84        2.95   9390.75
Cleric     131        3.24  14445.93
Rogue      176        3.32  19524.84
Mage       224        3.39  26730.00
Ranger     215        3.46  27188.02
Warrior    364        3.49  48870.51

Bards' average difficulty: 2.95
Warriors' average difficulty: 3.49

Wren Hawkridge: 16 quests, 2,289.67 cr
         quest_id
outcome
Failed          4
```

```
Fled            1
Success        11
```

Hint: It's true about the Bards. In Captain Siv's defence: the Bards asked. And Wren is a legend with a 69 % success rate; legends get the notices nobody else will take.

Bug hunt Pages from the Grimoire, page 10

15 min · `bughunt/war_table.py`

Found pinned under the big map on the War Table. It's every region at a glance for Captain Siv: quests, success rate and the warden. **Three** bugs: one crashes (the error mentions a "level name"; think about how groupby takes two keys), one quietly loses 90 quests, one gets every percentage slightly wrong (check one by hand against Exercise 4). One commit per bug, each message explaining what was wrong and why.

EXPECTED OUTPUT ONCE FIXED

```
Quests on the war table: 400
outcome         quests  success_pct                   warden
region
Thornwood           90         71.1              (no warden)
Saltmarsh           82         69.5   Warden Nerys Saltmarsh
Northern Wilds      75         64.0  Warden Corwin Northwind
Ironhills           85         61.2  Warden Hild Stonebridge
Greyfen             68         55.9        Warden Alder Pike
```

More riddles. Five more torn pages wait in `bughunt/riddles/`, one bug each (the last one has two), getting harder as you go. Optional: for fast finishers, or a quiet evening.